



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

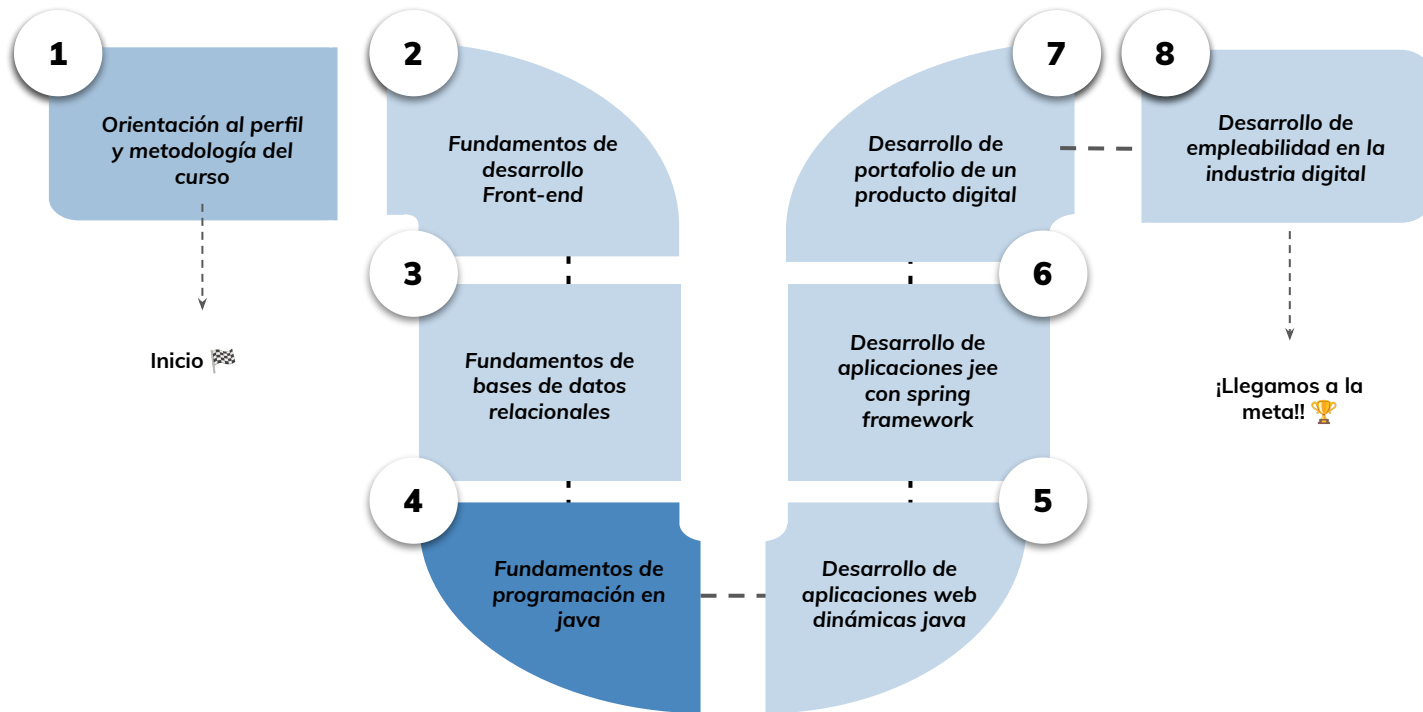


› El entorno java para la programación algoritmos- Parte 1

Aprendizaje Esperado 3: Utilizar la sintaxis básica del lenguaje java para la construcción de programas que resuelven un problema de baja complejidad.

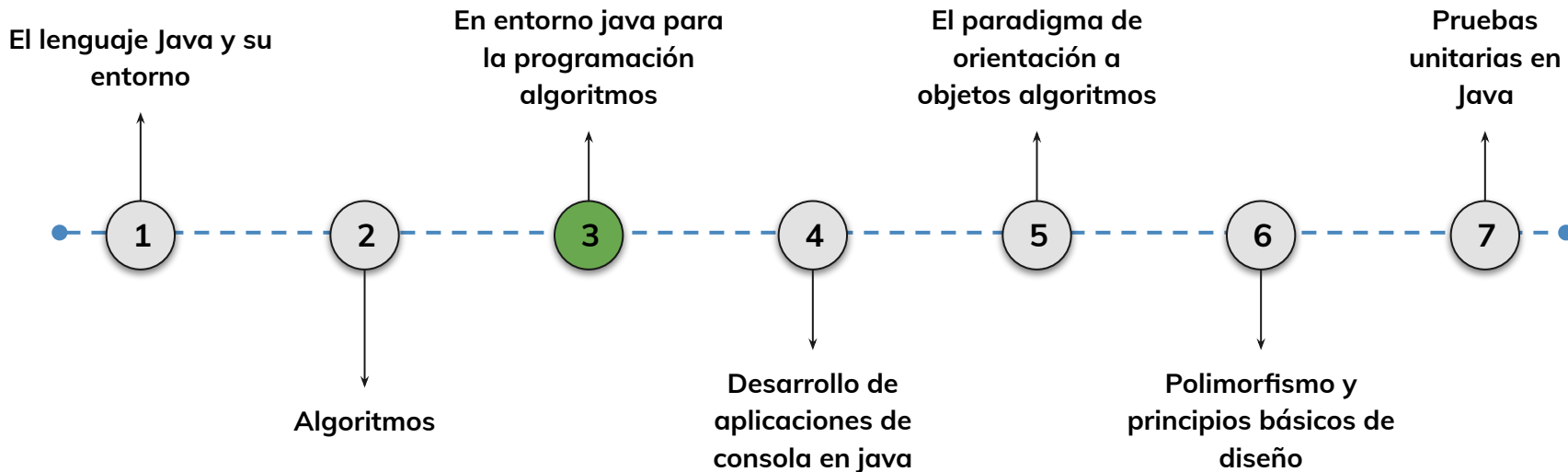
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

3.

Comprender la estructura básica de un programa Java con variables, tipos de datos e instrucciones condicionales

Durante esta clase aprenderemos a declarar variables en Java, utilizar tipos de datos primitivos, capturar datos desde teclado y aplicar instrucciones condicionales como if y else. Estos elementos son esenciales para el control del flujo en cualquier algoritmo.

Tipos de datos y variables en Java

Tipos primitivos: int, double, boolean, char

Declaración, inicialización y uso de variables

Entrada y condicionales

Lectura desde teclado con Scanner

Sentencias condicionales if, else, y operadores lógicos



Objetivos de aprendizaje

¿Qué aprenderás?



- Reconocer la implementación de las variables en Java
- Comprender el alcance de los tipos de datos en Java
- Reconocer la implementación y manipulación de los operadores en Java
- Comprender el concepto de las expresiones
- Implementar sentencias condicionales para controlar el flujo de un programa Java.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Reconocimos la sintaxis e implementación de las variables en Java.
- Comprendimos los tipos de datos y su manipulación.
- Conocimos e implementamos el uso de operadores en programas Java.
- Reconocimos la estructura de las expresiones.
- Reconocimos las sentencias condicionales

➤ Variables en Java

< > Variables

¿Qué características tienen las variables en Java?

Como ya hemos visto, las variables son contenedores con nombres que se utilizan para representar valores en un programa. Cada variable tiene un tipo de dato que define qué tipo de valores puede contener y cuánta memoria se reserva para almacenar esos valores.

Java es un lenguaje de **tipado estático**, por lo cual **todas las variables deben ser declaradas antes de ser utilizadas**. Estas tendrán un tipo de dato definido (ya sea un tipo de dato primitivo o una clase) y un nombre de identificador. El tipo de dato se asignará a la hora de definir la variable.



< > Variables

¿Cómo se declaran las variables en Java?

Una variable se declara especificando su tipo de dato y su nombre.

Inicialización de variables:

Una variable puede ser declarada y luego inicializada con un valor. La inicialización se realiza utilizando el operador de asignación (=).

Es posible declarar e inicializar en una misma línea, como vemos en la imagen.

```
//declaración de variables  
int numero = 2;  
String cadena = "Hola";  
double decimal = 2.4;  
boolean flag = true;
```



< > Variables

Alcance de las variables:

El alcance de una variable se refiere a la parte del programa donde la variable es accesible y puede ser utilizada. Las variables pueden tener alcance local (limitado a un bloque de código) o alcance de clase (disponible en toda la clase). Por ejemplo:

```
public class Ejemplo {  
    // Variable de alcance de clase  
    int variableClase;  
  
    public void miMetodo() {  
        // Variable de alcance local  
        int variableLocal = 10;  
        // ...  
    }  
}
```

< > Variables

Convenciones de nomenclatura

En Java, es común seguir convenciones de nomenclatura para nombrar variables de manera legible. Por lo general, los nombres de variables comienzan con una letra minúscula y utilizan el estilo camelCase para las palabras compuestas. Por ejemplo:

- edad
- alturaPersona
- contadorTotal



< > Variables

Antes de comenzar a practicar en nuestro IDE, ¿Qué es el método “main”?

El método **main** en Java es el punto de entrada de cualquier programa Java. Es el método que el sistema ejecuta cuando se lanza una aplicación. Sus principales características son:

- Es un método estático, lo que significa que puede ser llamado sin necesidad de instanciar la clase donde fue declarado.
- Su firma siempre debe ser: **public static void main(String[] args)**
- El nombre del método debe ser exactamente "**main**" para poder ejecutarse.
- Todo el código del programa se escribe dentro de este método, o se invoca desde él.

En éste método es donde comenzaremos las primeras pruebas de desarrollo en nuestros proyectos Java.





Ejercicio N° 1

< > Variables en Java

Contexto: 🙌

Vamos a comenzar a manipular las variables en Java. Para realizar este ejercicio es necesario que tengas instalado el IDE Eclipse, y que trabajes en la clase **main**.

Consigna: 📝

1. Declara una variable de tipo **char** llamada "identificador" y asígnale el dígito verificador de tu RUT. Imprime esta variable.
2. Crea una variable de tipo **boolean** llamada "estudiante" y asígnale true si eres estudiante, false en caso contrario. Imprime esta variable.
3. Declara una variable **String** "apellido" y asígnale tu apellido. Imprime esta variable concatenada con tu nombre.

Tiempo 🕒: 15 minutos



➤ Tipos de Datos

< > Tipos de datos

Tipos de datos primitivos:

Como ya hemos comentado Java es un lenguaje de tipado estático. Es decir, se define el tipo de dato de la variable a la hora de definir esta.

- **byte**; valores numéricos de -128 a 127 (ambos inclusive).
- **short**: valores numéricos de -32.768 a 32.767
- **int**: valores numéricos de -2,147,483,648 a 2,147,483,647.
- **long**: desde -9,223,372,036,854,775,808 a 9,223,372,036,854,775,807.
- **float**: conocido como tipo de precisión simple. De 1.4E-45 a 3.4028235E38.



< > Tipos de datos

Tipos de datos primitivos:

- **double** : en el rango de 4.9E-324 a 1.7976931348623157E308.
- **boolean** : Sirve para definir tipos de datos que tienen un valor de **true** o **false**.
- **char** : Es un tipo de datos que representa a un carácter Unicode sencillo de 16 bits.
Los datos de tipo caracter en Java se escriben entre comillas simples: 'A'

Es importante saber que estos son tipos de datos del lenguaje y que no representan objetos. Cosa que sí sucede con el resto de elementos del lenguaje Java.



< > Tipos de datos

Tipos de datos String:

Además de los tipos de datos primitivos el lenguaje de programación Java provee también un soporte especial para **cadenas de caracteres** a través de la clase String.

Encerrando la cadena de caracteres con **comillas dobles** se creará de manera automática una nueva instancia de un objeto tipo String.

```
String cadena = "Hola";
```

Los objetos String son **inmutables**, esto significa que una vez creados, sus valores no pueden ser cambiados. Si bien esta clase no es técnicamente un tipo de dato primitivo, el lenguaje le da un soporte especial para que actúe como tal.



Variables de caracteres y cadenas:

Veamos algunos ejemplos prácticos sobre cómo declarar variables en Java. Para esto, vamos a trabajar en el IDE Eclipse.

1. Declarar una variable entera llamada "edad" y asignarle un valor inicial. Mostrar el valor de la variable edad con `println`.
2. Declarar una variable de tipo `String` llamada "nombre" y asignarle un valor inicial. Imprimir el contenido de la variable nombre.
3. Declarar una variable de tipo `char` llamada "caracter" y asignarle un valor inicial. Mostraremos el valor de la variable por consola.



LIVE CODING

Ejemplo en vivo

4. Declarar e inicializar los tipos de datos enteros: byte, short, int, long.
5. Declarar e inicializar los tipos de datos para decimales: float y double.
6. Explicar el tipo de dato char para almacenar caracteres.
7. Explicar el tipo String para cadenas de texto.

Tiempo total: 30 minutos



Ejercicio N° 2

< > Variables en Java

Contexto: 🙌

Vamos a comenzar a manipular los tipos de datos numéricos en Java. Para realizar este ejercicio es necesario que tengas instalado el IDE Eclipse, y que trabajes en la clase **main**.

Consigna: ✍️

1. Declara variables de tipo **byte**, **short**, **int**, **long** y asígnale un valor entero a cada una. Imprime estas variables.
2. Crea variables **float** y **double** para números con decimales. Imprime estas variables.

Tiempo 🕒: 15 minutos



➤ Operadores en Java

< > Operadores en Java

¿Qué son los operadores?:

En programación un operador representa un símbolo que permite realizar operaciones aritméticas, relacionar elementos o hacer preguntas donde se involucra más de una condición.

Un **operador lleva a cabo operaciones** sobre uno (operador unario), dos (operador binario) o tres (operador ternario) datos u operandos de tipo primitivo devolviendo un valor determinado también de un tipo primitivo. El tipo de valor devuelto tras la evaluación depende del operador y del tipo de los operandos. Por ejemplo, los operadores aritméticos trabajan con operandos numéricos, llevan a cabo operaciones aritméticas básicas y devuelven el valor numérico correspondiente.



< > Operadores en Java

Operadores aritméticos

Los operadores aritméticos en Java son aquellos que nos permiten realizar operaciones matemáticas: **suma**, **resta**, **multiplicación**, **división** y **resto**.

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
-	operador unario de cambio de signo	-4	-4
+	Suma	2.5 + 7.1	9.6
-	Resta	235.6 - 103.5	132.1
*	Producto	1.2 * 1.1	1.32
/	División (tanto entera como real)	0.050 / 0.2 7 / 2	0.25 3
%	Resto de la división entera	20 % 7	6



< > Operadores en Java

Operadores unarios

Los operadores unarios en Java son aquellos que solo requieren un operando para funcionar.

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
++	Incremento i++ primero se utiliza la variable y luego se incrementa su valor ++i primero se incrementa el valor de la variable y luego se utiliza	4++ a=5; b=a++; a=5; b=++a;	5 a vale 6 y b vale 5 a vale 6 y b vale 6
--	decremento	4--	3



< > Operadores en Java

Operadores de relación

Realizan comparaciones entre datos compatibles de tipos primitivos (numéricos, carácter y booleanos) teniendo siempre un resultado booleano. Los operandos booleanos sólo pueden emplear los operadores de igualdad y desigualdad.

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
<code>==</code>	igual que	<code>7 == 38</code>	<code>false</code>
<code>!=</code>	distinto que	<code>'a' != 'k'</code>	<code>true</code>
<code><</code>	menor que	<code>'G' < 'B'</code>	<code>false</code>
<code>></code>	mayor que	<code>'b' > 'a'</code>	<code>true</code>
<code><=</code>	menor o igual que	<code>7.5 <= 7.38</code>	<code>false</code>
<code>>=</code>	mayor o igual que	<code>38 >= 7</code>	<code>true</code>



< > Operadores en Java

Operadores lógicos

Realizan operaciones sobre datos booleanos y tienen como resultado un valor booleano.

Operador	Descripción	Ejemplo de expresión	Resultado del ejemplo
!	Negación - NOT (unario)	<code>!false</code> <code>!(5==5)</code>	<code>true</code> <code>false</code>
	Suma lógica – OR (binario)	<code>true false</code> <code>(5==5) (5<4)</code>	<code>true</code> <code>true</code>
^	Suma lógica exclusiva – XOR (binario)	<code>true ^ false</code> <code>(5==5) (5<4)</code>	<code>true</code> <code>true</code>
&	Producto lógico – AND (binario)	<code>true & false</code> <code>(5==5) & (5<4)</code>	<code>false</code> <code>false</code>
	Suma lógica con cortocircuito: si el primer operando es <code>true</code> entonces el segundo se salta y el resultado es <code>true</code>	<code>true false</code> <code>(5==5) (5<4)</code>	<code>true</code> <code>true</code>
&&	Producto lógico con cortocircuito: si el primer operando es <code>false</code> entonces el segundo se salta y el resultado es <code>false</code>	<code>false && true</code> <code>(5==5) && (5<4)</code>	<code>false</code> <code>false</code>

< > Operadores en Java

Operadores lógicos

Para mejorar el rendimiento de ejecución del código **es recomendable** emplear en las expresiones booleanas el **operador &&** en lugar del operador &. En este caso es conveniente situar la condición más propensa a ser falsa en el término de la izquierda.

Esta técnica puede reducir el tiempo de ejecución del programa.

De forma equivalente es preferible emplear el **operador ||** al operador |. En este caso es conveniente colocar la condición más propensa a ser verdadera en el término de la izquierda.





LIVE CODING

Ejemplo en vivo

Practicando operaciones:

Vamos a realizar diversas operaciones en Java con Eclipse, y vamos a imprimir por pantalla los resultados de dichas operaciones:

1. Desarrollar operaciones aritméticas simples (suma, resta, multiplicación y división).
2. Desarrollar operaciones de incremento, utilizando el operador ++.
3. Desarrollar operaciones lógicas, utilizando && y ||

Tiempo: 25 minutos

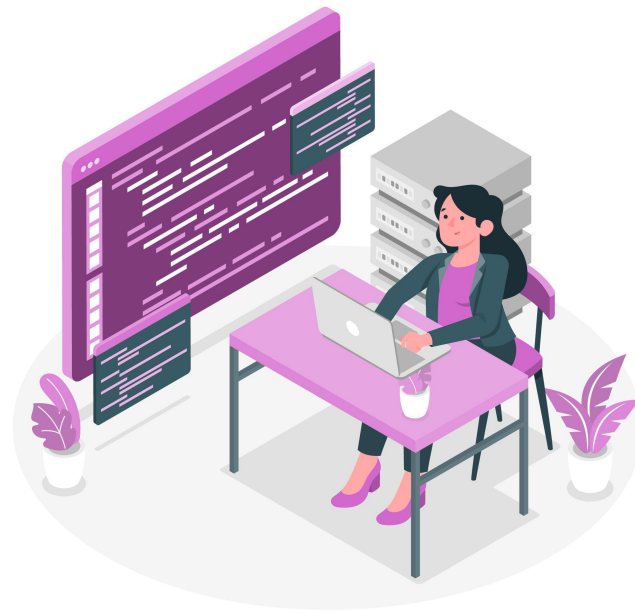
➤ Expresiones en Java

< > Expresiones

¿Qué es una expresión?

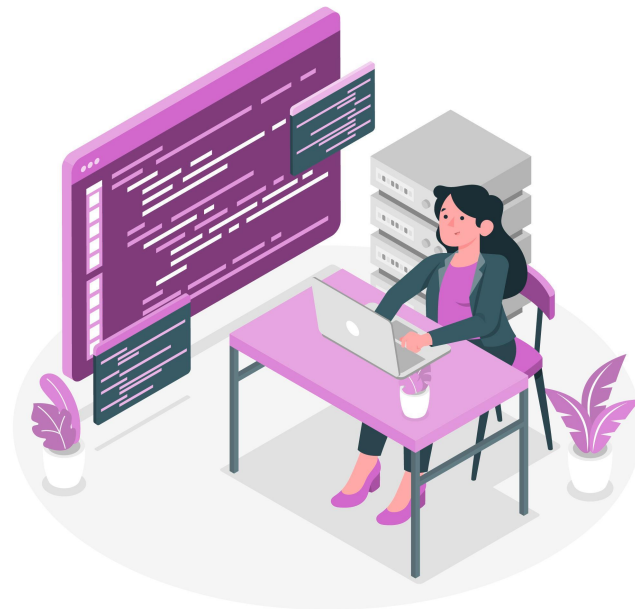
Una expresión es un conjunto de variables, operadores e invocaciones de métodos que se construyen para poder ser evaluadas retornando un resultado.

Cuando tengamos expresiones de evaluación complejas es recomendable que utilicemos paréntesis para saber cuál es el orden de ejecución de operaciones. En el caso de no utilizar paréntesis se ejecutará el orden de preferencia de operadores. Por ejemplo, la división tiene más preferencia que la suma.



< > Expresiones

Si dos operadores se encuentran en la misma expresión, el orden en el que se evalúan puede determinar el valor de la expresión. En la siguiente tabla se muestra el **orden o prioridad en el que se ejecutan los operadores** que se encuentren en la misma sentencia. Los operadores de la misma prioridad se evalúan de izquierda a derecha dentro de la expresión.



Prior.	Operador	Tipo de operador	Operación
1	++ -- +, - ~ i	Aritmético Aritmético Aritmético Integral Booleano	Incremento previo o posterior (unario) Incremento previo o posterior (unario) Suma unaria, Resta unaria Cambio de bits (unario) Negación (unario)
2	(tipo)	Cualquiera	
3	*, /, %	Aritmético	Multipliación, división, resto
4	+, - +	Aritmético Cadena	Suma, resta Concatenación de cadenas
5	<< >> >>>	Integral Integral Integral	Desplazamiento de bits a izquierda Desplazamiento de bits a derecha con inclusión de signo Desplazamiento de bits a derecha con inclusión de cero
6	<, <= >, >= instanceof	Aritmético Aritmético Objeto, tipo	Menor que, Menor o igual que Mayor que, Mayor o igual que Comparación de tipos
7	== i= == i=	Primitivo Primitivo Objeto Objeto	Igual (valores idénticos) Desigual (valores diferentes) Igual (referencia al mismo objeto) Desigual (referencia a distintos objetos)
8	& &	Integral Booleano	Cambio de bits AND Producto booleano
9	^ ^	Integral Booleano	Cambio de bits XOR Suma exclusiva booleana
10	 	Integral Booleano	Cambio de bits OR Suma booleana
11	&&	Booleano	AND condicional
12		Booleano	OR condicional
13	? :	Booleano, cualquiera, cualquiera	Operador condicional (ternario)
14	= *=, /=, %= +=, -= <<=, >>= >>>= &=, ^=, =	Variable, cualquiera	Asignación Asignación con operación

➤ Sentencias y Bloques en Java

< > Sentencias

¿Qué es una sentencia?:

Una sentencia es la **unidad mínima de ejecución de un programa** . Un programa se compone de uno o varios conjuntos de sentencias que acaban resolviendo un problema. En Java, **al final de cada una de las sentencias encontraremos un punto y coma (;)**.

- Sentencias de **declaración** : `int valor = 2;`
- Sentencias de **asignación** : `valor = 2;`
- Sentencias de **incremento** o **decremento** : `valor++;`
- Invocaciones a **métodos** : `System.out.println("Hola Mundo");`
- Creaciones de **objetos** : `Circulo miCirculo = new Circulo();`
- Sentencias de **control de flujo** : `if (valor>1) { ... };`



< > Bloques

¿Qué es un bloque en Java?:

Cuando hablamos de bloque, en programación, nos referimos a un **trozo de código que está delimitado** de alguna forma. **En Java, son unas llaves { }** . Una de apertura al principio y otra de cierre al final. Así, indicamos donde empieza ese bloque de código y dónde termina.

Los bloques son conjuntos de sentencias delimitados.



< > Bloques

Analizamos el código:

Las clases pueden contener entre sus llaves, muchas cosas, entre ellas, métodos, que es lo que empieza en la línea 2. Este método llamado main, es el encargado de ejecutar las instrucciones que hay en las líneas 3, 4, y 5.

El método, al igual que pasa con la clase, se abre en la línea donde se declara (línea 2) con una llave de apertura y se cierra cuando acaban las instrucciones que lleva escritas con la llave de cierre (línea 6).

El método es un bloque de código que está dentro de la clase, por este motivo, está envuelto entre las primeras llaves.

```
1 public class Main {  
2     public static void main(String[] args) {  
3         int numero1 = 7;  
4         int resultado = 10 + numero1;  
5         System.out.print(resultado);  
6     }  
7 }
```



LIVE CODING

Ejemplo en vivo

Practicando:

Vamos a practicar un código simple para poner a prueba la jerarquía de las operaciones. Levantando la mano, los invitamos a contestar **¿Cuál operación se ejecuta primero?** **¿Pueden dar otro ejemplo?**

1. `int resultado = 2 + 4 * 5;`
2. `int resultado = 2 + (4 * 5);`
3. `int resultado = 5 * 4 / 3;`
4. `int resultado = 5 * (4 + 3);`

Tiempo: 15 minutos

› Sentencias condicionales en Java

< > Condicionales

¿Qué son las sentencias condicionales?:

Los condicionales son **estructuras de control que cambian el flujo de ejecución de un programa de acuerdo a si se cumple o no una condición** . Cuando el flujo de control del programa llega al condicional, el programa evalúa la condición y determina el camino a seguir. En Java existen dos tipos de estructuras condicionales, las estructuras **if / else** (si-entonces) y la estructura **switch** (según).



➤ Condicional If

< > IF

Sintaxis en Java

La estructura if es la más básica de las estructuras de control de flujo. Esta estructura le indica al programa que ejecute determinado bloque del código si y sólo si la condición evaluada es verdadera («true»).

La forma más simple de esta estructura es la siguiente:

```
if (<condición>) {  
    <sentencias>  
}
```

En esta estructura, <condición> es una expresión condicional cuyo resultado luego de la evaluación es un dato booleano(lógico) verdadero o falso. El bloque de instrucciones <sentencias> se ejecuta si y sólo si la expresión se evalúa como true, es decir, se cumple la condición.





IF - ELSE

Sintaxis en Java

En caso de que la condición no se cumpla y se necesite ejecutar otro bloque de código para estos casos, otra forma de expresar esta estructura es la siguiente:

```
if(<condición>){  
    <sentencias A>  
} else {  
    <sentencias B>  
}
```

El flujo de control del programa funciona de la misma manera, cuando se ejecuta la estructura if, se evalúa la expresión condicional, si el resultado de la condición es verdadero se ejecutan las sentencias que se encuentran contenidas dentro del bloque de código if (<sentencias A>).

Si la evaluación da como resultado false, se ejecutan las sentencias contenidas dentro del bloque else (<sentencias B>).



< > IF - ELSE - anidados

Sintaxis en Java

En muchas ocasiones, se anidan estructuras alternativas if-else, de forma que se pregunte por una condición si anteriormente no se ha cumplido otra y así sucesivamente.

```
if (<condicion1>) {  
    <sentencias A>  
} else if (<condicion2>){  
    <sentencias B>  
} else {  
    <sentencias C>  
}
```



LIVE CODING

Ejemplo en vivo

Condicionales IF:

Veamos un ejemplo de cada condicional IF en práctica, representando las siguientes directivas. Vamos a codear 3 sentencias distintas:

1. Si es mayor a 18, imprimir la edad por pantalla.
2. Si es mayor a 18, imprimir la edad por pantalla, si es menor a 18, imprimir un mensaje que indique que es menor.
3. Si es mayor a 18, imprimir la edad por pantalla. Si es igual a 18, que imprima "tiene 18 actualmente". Si es menor, que lo indique por pantalla.

Tiempo: 20 minutos



Ejercicio N° 1

< > Condicionales if

¡A practicar! 🙌

Para este ejercicio, vamos a crear 2 sentencias condicionales distintas. Cada una debe resolver y mostrar por pantalla determinado resultado. También aprovecharemos este ejercicio para practicar operadores y reglas aritméticas.

Consigna: 📝

1. Escribir un IF que evalúe si una variable número es **positiva** .
2. Escribir un IF-ELSE que evalúe si un número es **par o impar** . Debes devolver un mensaje por pantalla para cada caso. Si el número ingresado es **cero** , debes mostrar por pantalla un mensaje informándolo. Investiga el uso del operador **MOD en Java (%)** .

Tiempo 🕒: 30 minutos



➤ Condicional Switch

< > Switch

Sintaxis en Java

El bloque switch evalúa qué valor tiene la variable, y de acuerdo con el valor que posee ejecuta las sentencias del bloque case correspondiente.

```
switch( <variable>){  
  case <valor1>:  
    <sentencias1>  
  break;  
  case <valor2>:  
    <sentencias2>  
  break;  
  default:  
    <sentencias3>  
}
```



< > Switch

Sentencia break:

El uso de la sentencia **break** que va luego de cada **case** tiene la función de **detener la sentencia switch** que la envuelve, es decir que el control de flujo del programa continúa con la primera sentencia que se encuentra a continuación del cierre del bloque switch.

Si el programa comprueba que se cumple el primer valor (valor1) se ejecutará el bloque de instrucciones <sentencias1>.

Las instrucciones dentro del bloque **default** se ejecutan cuando la variable que se está evaluando **no coincide con ninguno de los valores case** . Esta sentencia equivale al else de la estructura if-else.





LIVE CODING

Ejemplo en vivo

Condicionales IF:

Veamos un ejemplo del condicional SWITCH en práctica.

1. *Desarrollemos un condicional switch que evalúe un número ingresado del 1 al 12 y nos devuelva por pantalla a cuál mes del año hace referencia.*

Tiempo: 15 minutos



Momento:

Time-out!



5 -10 min.



¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ Reconocimos la sintaxis e implementación de las variables en Java.
- ✓ Comprendimos los tipos de datos y su manipulación.
- ✓ Conocimos e implementamos el uso de operadores en programas Java.
- ✓ Reconocimos la estructura de las expresiones.
- ✓ Reconocimos las sentencias condicionales



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

